— *Project Report* —

Computational Geometry *and* Digital Images

Hugo SALOU

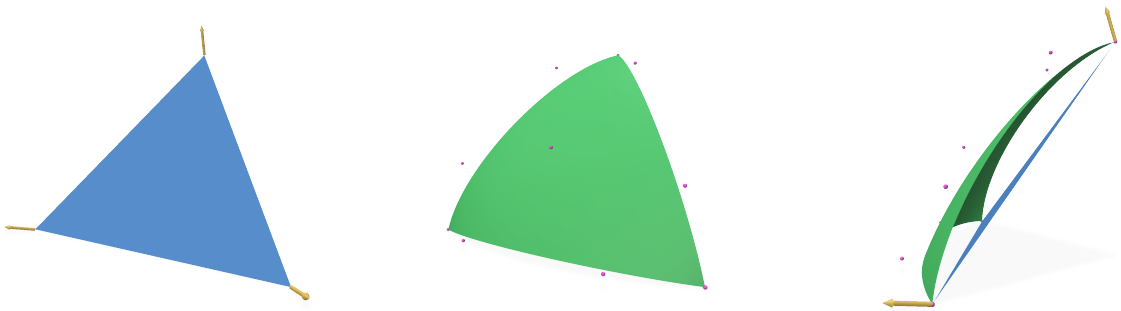
1 Context of this project.

Three dimensional meshes are based on a finite set of triangles. The smaller this set, the faster we are able to do operate on that 3D object (applying some transformation on the vertices, displaying it, doing physical simulations, *etc*). It is thus crucial to keep a low number of triangles in, for example, real-time applications. Lowering the number of triangles also allows for a lower memory footprint for assets. This idea is closely related to *procedural generation* of 3D models, textures, *etc*: trading "ahead-of-time" generation for "run-time" generation. This generation at run-time is more costly that simply reading pre-computed results from a file. However, in some cases, the run-time computation can be done in parallel, on the GPU.

2 Presentation of curved PN triangles [Vla+01].

This project is based on [Vla+01]. They provide an algorithm to take a 3D mesh with a small number of triangles, and turn in into a finer, more detailed mesh. This initial mesh must contain normal vectors on each vertex, instead of each triangle. This is called a *point-normal (PN) triangle mesh*.

Each PN triangle is then *curved* (hence the name of the paper) in coherence with the direction of the normal vectors at the triangle's vertices. This "curving" is done using a cubic Bézier patch (figure 1, page 1).



(a) *Initial PN triangle* (b) *Generated Bézier patch* (c) *Side view of the patch*

Figure 1 | *A PN triangle and its associated curved PN triangle.*

A Bézier patch is given as the following parametric surface

$$\vec{b}: \begin{array}{rcl} \Delta & \longrightarrow & \mathbb{R}^3 \\ (u,v,w) & \longmapsto & \sum_{i+j+k=3} \vec{b}_{kij}\, u^i v^j w^k, \end{array} \qquad \text{where} \quad \Delta := \left\{ (u,v,w) \;\middle|\; \begin{array}{l} u,v,w \geq 0 \\ u+v+w=1 \end{array} \right\}.$$

Given a PN triangle whose vertices' positions are $\vec{P}_1, \vec{P}_2, \vec{P}_3$ and vertices' normals are $\vec{N}_1, \vec{N}_2, \vec{N}_3$, we define the values of control points $\vec{b}_{kij}$ (shown in magenta in figure 1) as follow

$$\begin{aligned} \vec{b}_{300} &:= \vec{P}_1, & \vec{b}_{030} &:= \vec{P}_2, & \vec{b}_{003} &:= \vec{P}_3, \\ \vec{b}_{012} &= \frac{1}{3}(2\vec{P}_3 + \vec{P}_2 - w_{32}\vec{N}_3), & \vec{b}_{021} &= \frac{1}{3}(2\vec{P}_2 + \vec{P}_3 - w_{23}\vec{N}_2), \\ \vec{b}_{102} &= \frac{1}{3}(2\vec{P}_3 + \vec{P}_1 - w_{31}\vec{N}_3), & \vec{b}_{201} &= \frac{1}{3}(2\vec{P}_1 + \vec{P}_3 - w_{13}\vec{N}_1), \\ \vec{b}_{120} &= \frac{1}{3}(2\vec{P}_2 + \vec{P}_1 - w_{21}\vec{N}_2), & \vec{b}_{210} &= \frac{1}{3}(2\vec{P}_1 + \vec{P}_2 - w_{12}\vec{N}_1), \end{aligned}$$

$$\vec{b}_{111} := \alpha\vec{E} + (1-\alpha)\vec{V},$$

where

- ▷ $w_{ij} := (\vec{P}_j - \vec{P}_i) \cdot \vec{N}_i$,
- ▷ $\vec{E} := \frac{1}{6}(\vec{b}_{012} + \vec{b}_{021} + \vec{b}_{102} + \vec{b}_{201} + \vec{b}_{120} + \vec{b}_{210})$,
- ▷ $\vec{V} := \frac{1}{3}(\vec{P}_1 + \vec{P}_2 + \vec{P}_3)$.

Changing the value of α changes how "bumpy" the triangle gets (figure 2). In [Vla+01], the value of α is set to $1/2$. Figure 2 shows curved PN triangles obtained with different values for α (sliced view in the middle of triangle). In figure 2, the "most bumpy" (outwards) PN triangle, in yellow, corresponds to $\alpha = 2$. Other values are, in order,

$$\alpha = 1.5, \quad 1.0, \quad 0.75, \quad 0.5, \quad 0.25, \quad 0.0.$$

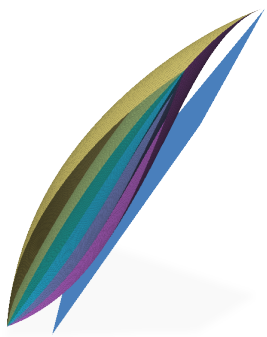


Figure 2 | *Influence of α.*

Note that choosing $\alpha < 1$ leads to a triangle that "curves inwards" in its center, while its edges still curve outwards (in pink on the figure).

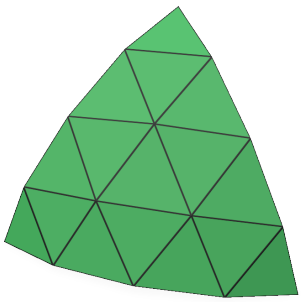


Figure 3 | *Result of the tessellation*

After computing the control points, it suffices to consider a tessellation of the triangle Δ into sub-triangles, and compute the vertices of every sub-triangle using $\vec{b}$. This allows for the use of a "level of details" parameter that can be changed if needed.

One major advantage of curved PN triangles over some similar methods is that, when computing a curved PN

triangle, we only need to consider one triangle. This allows the use of the GPU for computing curved PN triangles and their tessellation using, for example, a geometry shader in OpenGL.

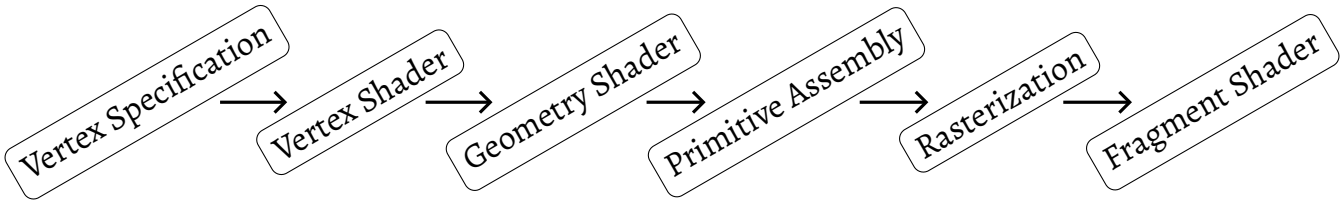


Figure 4 | *OpenGL's rendering pipeline (simplified)*

A *geometry shader* in OpenGL (figure 4) is a program that gets a triangle and transform it into some geometry (hence the name *geometry* shader), eventually emitting multiple triangles. A geometry shader for producing curved PN triangles from PN triangles was implemented in less than 200 lines of GLSL (`shaders/shader.geom`).

3 Comparing different methods for computing normals.

After computing the curved PN triangles and their tessellation, it is important to compute normals for the vertices of the new triangles, so that they can be passed to the fragment shader after the rasterization.

In [Vla+01], they define *quadratic varying normals*, a sort of quadratic Bézier patch (but representing normals and not positions). Given a triangle, they define

$$\vec{n}(u,v,w) := \text{normalized}\Big(\sum_{i+j+k=2} \vec{n}_{kij} u^i v^j w^k\Big),$$

where Bézier control points $\vec{n}_{kij}$ are defined similarly to $\vec{b}_{kij}$.

Another way of computing normals would be *linear varying normals* (also given in [Vla+01]): for a triangle whose normals are $\vec{N}_1$, $\vec{N}_2$ and $\vec{N}_3$, define

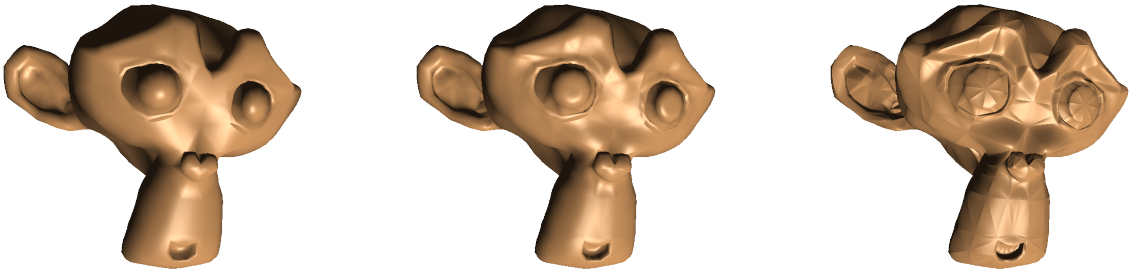
$$\vec{n}(u,v,w) := \text{normalized}\big(u\vec{N}_1 + v\vec{N}_2 + w\vec{N}_3\big).$$

A third way of computing the normals would be "*exact*" in the sense that it would follow exactly from the parametric surface $\vec{b}(u,v)$, but would not use directly the normals provided by the 3D model:

$$\vec{n}(u,v,w) := \text{normalized}\left(\frac{\partial\vec{b}(u,v,w)}{\partial u} \times \frac{\partial\vec{b}(u,v,w)}{\partial v}\right),$$

where $- \times -$ denotes the cross product. There is no need to consider $\partial\vec{b}(u,v,w)/\partial w$ as it is not independent from u and v: $w = 1 - u - v$. Exact computations of the derivatives in

terms of the Bézier control points $\vec{b}_{kij}$ were done in Maple and results can be found in the shader file `shaders/shader.geom`.

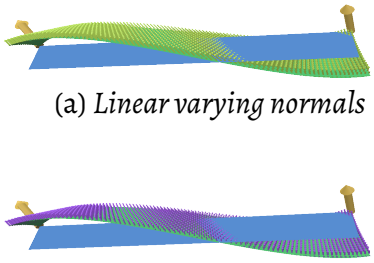


(a) *Linear varying normals* (b) *Quadratic varying normals* (c) *"Exact" normals*

Figure 5 | *Comparison of different normals-computing methods*

As can be seen in figure 5 (page 4), the three methods give similar results at the center of triangles. Yet, at the boundary, shading differs. When using Phong shading (as done in figure 5, *c.f.* `shaders/shader.frag`), linear varying normals seem to give better results.

However, as stated in [Vla+01], linear varying normals can sometimes oversimplify the variations of the surface, as shown in figure 6.



(a) *Linear varying normals*

(b) *Quadratic varying normals*

Figure 6 | *Normals on a varying surface*

4 Applications of curved PN triangles.

4.1 Marching cubes [LC87], procedural generation.

The *marching cubes algorithm*, introduced in [LC87], allows us to extract an isosurface of some given function. By design, this algorithm produces blocky-looking results. The use of curved PN triangles would allow for a smoother surface without too much overhead.

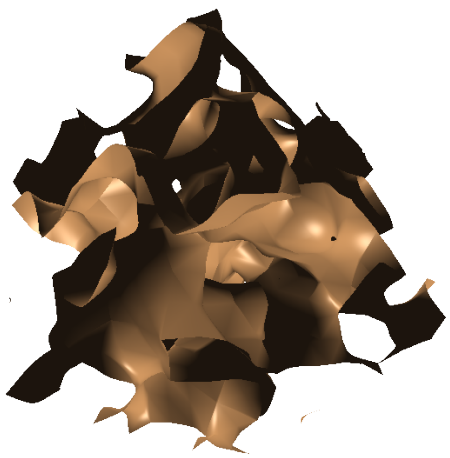


Figure 7 | *Marching cubes on Perlin noise rendered with curved PN triangles*

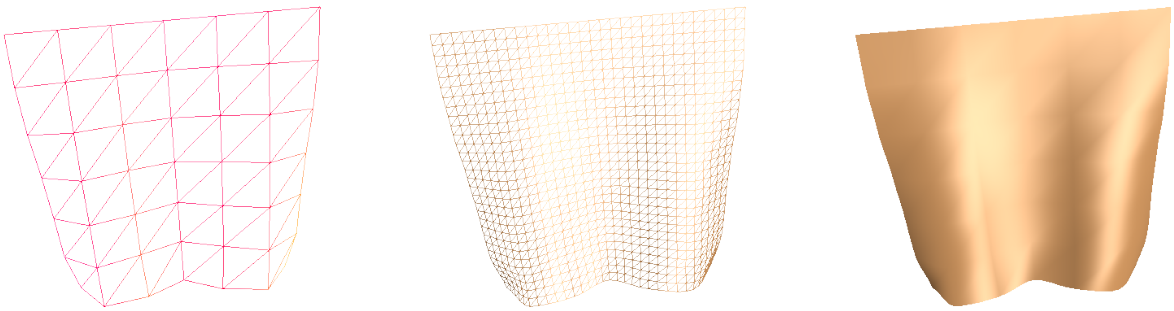
For examples, procedural cave generation can be done using the marching cube algorithm and some noise function like Perlin noise. In that case, no need to perform a lot of subdivisions on far-away parts of the cave. This can easily be done using curved PN triangles without having to run the marching cubes algorithm when a new level of details is needed. And, all the necessary data is already in the GPU's memory, thus no data-updates need to happen.

A basic example of curved PN triangles applied to marching cubes on some Perlin noise can be found in `cubes.py` and `opengl_cubes.py`.

4.2 Cloth simulations.

Consider the following cloth-simulation model: a grid in which every vertex is connected to its eight neighbors using springs. Thus, in this model, a vertex is only affected by the tension force of the springs and gravity.

Such a simple can be easily simulated (*c.f.* `cloth.py`), but this simulation can become very costly as the size of the mesh increases. One solution is to simulate on a coarse mesh and to use curved PN triangles for the rendering part (*c.f.* `opengl_cloth.py`). In games, for example, this could allow for real-time rendering without costing too much performance. This technique also enables the use of level of details, as discussed in subsection 4.1.



(a) *Cloth mesh* (b) *Render mesh* (c) *Resulting surface*

Figure 8 | *Simulation on a coarse mesh and curved PN triangles for rendering*

Even really small grids (7×7 in figure 8a) produce believable enough-results. Doing the simulation on the render mesh (75 times finer in the case of figure 8b) would result in way higher computation costs (about 5 000 times higher with the details shown in figure 8b). A video of the cloth simulation is available: `videos/cloth-simulation.mov`.

5 Conclusion.

Curved PN triangles are a pretty inexpensive way of smoothing a mesh as it can be done on the GPU using, for example, a geometry shader in OpenGL. It allows the use of algorithms on some coarser meshes which are then rendered in a higher resolution. Here, only marching cubes and cloths simulations were presented, but physical computations, *e.g.* finite element methods, could be done on PN triangles (acting on the position of vertices but also their normals) that are then subdivided when the strain gets too high to avoid loss of precision. This may not be the best option as curved PN triangles were designed for rendering in mind, so they may only *look* smooth, whereas other methods like Catmull-Clark subdivision [CC78] produces meshes that *are* smooth.

One important note is that meshes *can* be designed for curved PN triangles-based rendering [Wu+10]. PLY files in the `data/` folder were made using *Blender* without optimizing the mesh for this process in mind.

As one might suspect, curved PN triangles have some drawbacks. For example, even if the starting mesh is continuous, the resulting mesh can be discontinuous. This is usually the case when the mesh's normals are badly computed. Shading can also reveal the original mesh when the normals are not well-interpolated. Another drawback comes for the force of curved PN triangles: they smooth out the mesh, and thus, hard edges are also smoothed out. This can be fixed by changing the mesh or by updating a few terms as discussed in [Vla+01].

Curved PN *quads* is something that was introduced in [Pet08] and more work was done in [Pap14]. They allow for more control points, but the OpenGL pipeline does not support quads as a primitive.

Code related to this project can be found on GitLab:

`https://gitlab.aliens-lyon.fr/hsalou/m1-cgdi-project`.

References

[CC78] Edwin E. Catmull and J. Clark. "Recursively generated B-spline surfaces on arbitrary topological meshes". In: *Seminal graphics: pioneering efforts that shaped the field* (1978). URL: `https://api.semanticscholar.org/CorpusID:121149868`.

[LC87] William E. Lorensen and Harvey E. Cline. "Marching cubes: A high resolution 3D surface construction algorithm". In: *SIGGRAPH Comput. Graph.* 21.4 (Aug. 1987), pp. 163–169. ISSN: 0097-8930. DOI: `10.1145/37402.37422`. URL: `https://doi.org/10.1145/37402.37422`.

[Pap14] Chavdar Papazov. "Local, polynomial g 1 pn quads". In: *International Symposium on Visual Computing*. Springer. 2014, pp. 63–74.

[Pet08] Jörg Peters. *PN-quads*. Tech. rep. Tech. Rep. 2008-421, 2008.

[Vla+01] Alex Vlachos et al. "Curved PN triangles". In: *Proceedings of the 2001 Symposium on Interactive 3D Graphics*. I3D '01. New York, NY, USA: Association for Computing Machinery, 2001, pp. 159–166. ISBN: 1581132921. DOI: `10.1145/364338.364387`. URL: `https://doi.org/10.1145/364338.364387`.

[Wu+10] Xiaoqun Wu et al. "Simplification of Meshes into Curved PN Triangles". In: *Proceedings of the Second APSIPA Annual Summit and Conference*. 2010, pp. 598–604.